

Parella de Punts Més Propera i Més Llunyana

Pràctica 3

Serafí Nebot Ginard

Universitat de les Illes Balears (UIB)

Grau en Enginyeria Informàtica

Algorismes Avançats, Curs 2025–2026

Resum—Aquesta memòria descriu el disseny i la implementació d'una aplicació Java que resol el problema de la parella de punts més propera (*closest pair*) i més llunyana (*farthest pair*) sobre un conjunt de punts en el pla. S'implementen quatre algorismes: força bruta $O(n^2)$, divideix i venceràs $O(n \log n)$, una variant de divideix i venceràs amb optimització per *buckets* $O(n \log n)$, i la parella més llunyana mitjançant *convex hull* i *rotating calipers* $O(n \log n)$. L'aplicació segueix el patró MVC, ofereix una interfície gràfica interactiva amb visualització dels punts i resultats, i incorpora una eina de benchmark amb gràfiques i predicció de temps. Es presenta una anàlisi experimental rigorosa que confirma les complexitats teòriques i revela que l'optimització per *buckets* no aporta guany pràctic per al problema de la parella més propera.

Termes d'índex—parella més propera, parella més llunyana, divideix i venceràs, força bruta, convex hull, rotating calipers, Model-Vista-Controlador, benchmark, Java Swing

1. Introducció

El problema de la parella de punts més propera (*closest pair problem*) és un problema fonamental de la geometria computacional: donat un conjunt de n punts en el pla, trobar la parella de punts amb la distància euclidiana mínima. La solució per força bruta examina totes les $\binom{n}{2}$ parelles amb cost $O(n^2)$, però l'algorisme de divideix i venceràs resol el problema en $O(n \log n)$.

El problema complementari, la parella més llunyana (*farthest pair*), té la mateixa complexitat òptima $O(n \log n)$ però utilitza una estratègia diferent: primer es construeix el polígon convex que envolta tots els punts del núvol (*convex hull*) i després s'aplica la tècnica dels *rotating calipers*.

L'aplicació implementada permet generar núvols de punts amb quatre distribucions (uniforme, gaussiana, exponencial, agrupada), modificar paràmetres d'aquestes, executar els quatre algorismes, visualitzar els resultats gràficament i realitzar benchmarks sistemàtics amb predicció de temps.

2. Descripció de l'Aplicació

L'aplicació és una eina interactiva d'escriptori (Java Swing) que permet generar núvols de punts, trobar la parella més propera i més llunyana, i comparar el rendiment dels algorismes. La interfície es divideix en tres zones: un panell de controls a la part superior, la visualització de punts al centre i un panell de resultats a la dreta.

2.1. Generació de Punts

L'usuari configura el nombre de punts n amb un *spinner* i selecciona una de les quatre distribucions disponibles. Cada distribució té paràmetres configurables que es mostren dinàmicament:

- **Uniforme:** punts distribuïts uniformement dins del rang $[\min, \max]$ en ambdós eixos.
- **Gaussiana:** paràmetres de mitjana (fracció del rang), desviació estàndard σ (fracció del rang) i múltiples distribucions independents per a cada eix, permetent distribucions el·líptiques.
- **Exponencial:** factors λ_x i λ_y que controlen la densitat de punts cap a l'origen.
- **Agrupada (Clustered):** nombre de clusters (mínim 3), amb centres generats uniformement i punts distribuïts amb una gaussiana al voltant de cada centre.

Un temporitzador *debounce* de 300 ms evita regenerar els punts a cada tecleig: només es llança la generació quan l'usuari ha deixat d'interactuar durant 300 ms.

2.2. Visualització

El panell central (`PointPanel`) renderitza els punts amb `Graphics2D` i *antialiasing*. Les coordenades del domini es transformen a coordenades de pantalla amb un marge de 30 px. L'eix Y s'inverteix per respectar la convenció matemàtica (Y creix cap amunt). Quan s'han calculat els resultats, la parella més propera es ressaltava en vermell i la més llunyana en blau, amb una línia que connecta els dos punts de cada parella.

2.3. Execució dels Algorismes

El botó *Executar* llança els quatre algorismes en un fil de fons (*SwingWorker*) per no bloquejar la interfície. Durant l'execució, els controls es deshabiliten i el cursor canvia a mode d'espera. El botó *Aturar* permet cancel·lar l'execució cooperativament mitjançant `Thread.interrupt()`.

Cada algorisme s'executa amb el protocol de benchmark descrit a la Secció 3.8: 3 execucions d'escalfament descartades, seguides de 5 execucions mesurades. El temps mostrat al panell de resultats és la mitjana de les 5 execucions mesurades.

2.4. Finestra de Benchmark

El botó *Benchmark* obre una finestra independent que permet executar benchmarks sistemàtics amb N valors configurables. La finestra inclou:

- **Configuració:** valors de n separats per comes, distribució i paràmetres.
- **Gràfica:** visualització dels costos computacionals amb eixos, graella, llegenda i corbes ajustades. Permet escala logarítmica i visibilitat individual de cada sèrie.
- **Taula de dades:** mostra els temps per a cada algorisme.
- **Predicció:** ajusta constants multiplicatives als models teòrics ($a \cdot n^2$ per força bruta, $a \cdot n \log_2 n$ per D&C i la parella llunyana) i permet predir temps per a valors de n arbitraris.

3. Disseny i Implementació

3.1. Patró MVC i Comunicació per Events

L'aplicació segueix el patró Model-Vista-Controlador amb comunicació per events semàntics:

- 1) **Model** (Model): encapsula el núvol de punts, els resultats de cada algorisme (parella i temps), i els resultats del benchmark. No conté cap lògica d'algorisme; només emmagatzema dades.
- 2) **Vista** (MainView, PointPanel, BenchmarkWindow, BenchmarkGraphPanel, DistributionParamPanel): presenta els controls, la visualització i la gràfica. No manté estat de domini.
- 3) **Controlador** (Controller): implementa la interfície *ViewListener* per rebre els events de la vista, gestiona el cicle de vida dels fils de fons i actualitza el model i la vista.

La interfície *ViewListener* defineix quatre events: `onGenerate`, `onRun`, `onStop` i `onOpenBenchmark`. La vista emet events sense conèixer qui els processa, i el controlador implementa la interfície sense conèixer els components gràfics interns de la vista.

3.2. Interfície PairFinder i Patrons de Disseny

Tots els algorismes implementen la interfície *PairFinder*:

```
1 public interface PairFinder {  
2     PointPair find(Point[] points);  
3 }
```

Listing 1. Interfície comuna per a tots els algorismes.

Aquesta interfície permet tractar tots els algorismes de manera uniforme al benchmark i al controlador, seguint el **patró Strategy** [4]. A més, la relació entre *DivideConquerClosest* i *DivideConquerClosestBucket* utilitza el **patró Template Method**: la classe base defineix l'esquelet de l'algorisme (ordenar per X , recursió, construir strip) i la subclasse només sobreescriu el mètode `processStrip()` per canviar la forma de processar la franja central.

Aquesta decisió de disseny elimina la duplicació de codi: les dues variants comparteixen la recursió, la cerca binària de límits de la franja, i el cas base per força bruta. Només el processament de la franja (125 línies de codi eliminades) difereix entre les dues variants.

3.3. Algorisme de Força Bruta

L'algorisme de força bruta (*BruteForceClosest*) examina totes les $\binom{n}{2} = \frac{n(n-1)}{2}$ parelles de punts i retorna la de distància mínima. Comprova `Thread.interrupted()` a cada iteració del bucle extern per permetre la cancel·lació cooperativa.

```
1 for (int i = 0; i < n - 1; i++) {  
2     for (int j = i + 1; j < n; j++) {  
3         double dist = points[i].distanceTo(points[j]);  
4         if (dist < bestDist) {  
5             bestDist = dist;  
6             bestP1 = points[i];  
7             bestP2 = points[j];  
8         }  
9     }  
10 }
```

Listing 2. Nucli de l'algorisme de força bruta.

3.4. Algorisme de Divideix i Venceràs

L'algorisme de divideix i venceràs (*DivideConquerClosest*) segueix l'esquema clàssic de Shamos i Hoey [1]:

- 1) Ordenar els punts per coordenada x una sola vegada ($O(n \log n)$).
- 2) Recursivament trobar la parella més propera a la meitat esquerra i la meitat dreta.
- 3) Construir la franja central de punts amb $|x - x_{\text{mid}}| < d_{\text{min}}$.
- 4) Processar la franja: ordenar per y i per a cada punt, comparar amb els 7 veïns següents (argument de l'empaquetament).

El cas base s'activa quan el subproblema té 3 o menys punts, resolent-se per força bruta amb cost $O(1)$.

3.4.1. Optimització: Cerca Binària per als Límits de la Franja. En lloc d'un escaneig lineal $O(k)$ per trobar els punts dins de la franja, s'utilitza cerca binària $O(\log k)$ amb els mètodes `lowerBound` i `upperBound`, aprofitant que els punts ja estan ordenats per x . Això no millora la complexitat asimptòtica (el pas dominant segueix sent l'ordenació de la franja) però redueix el treball constant.

3.4.2. Optimització: Buffer Preal·locat per a la Franja. A cada nivell de recursió, l'algorisme necessita copiar els punts de la franja central a un buffer temporal per ordenar-los per y . La implementació ingènua crearia un nou array a cada crida recursiva, generant $O(\log n)$ al·locacions (una per nivell).

Per eliminar la pressió sobre el *garbage collector* i millorar la localitat de cache, s'al·loca un únic array `Point[n]` al mètode `find()` i es reutilitza a tots els nivells de recursió mitjançant `System.arraycopy`:

```
1 public PointPair find(Point[] points) {
2     // ...
3     Point[] strip = new Point[n]; // unica
        allocacio
4     return closestRec(sortedByX, 0, n - 1, strip);
5 }
```

Listing 3. Preal·locació del buffer de la franja.

El buffer és segur perquè cada nivell de recursió l'omple amb `System.arraycopy` i només accedeix als índexs $[0, k)$ on k és el nombre de punts de la franja actual. Quan la crida retorna, el contingut anterior ja no és necessari.

3.4.3. Patró Template Method per a la Franja. El mètode `processStrip` és `protected` i `virtual`, permetent que la subclasse `DivideConquerClosestBucket` el sobre-escrigui amb una estratègia diferent sense duplicar la resta de l'algorisme:

```
1 protected PointPair processStrip(
2     Point[] strip, int k,
3     double dMin, PointPair best) {
4     Arrays.sort(strip, 0, k,
5         Comparator.comparingDouble(Point::y));
6     for (int i = 0; i < k; i++) {
7         for (int j = i + 1; j < k
8             && (strip[j].y() - strip[i].y()) <
9                 dMin;
10             j++) {
11             double dist = strip[i].distanceTo(
12                 strip[j]);
13             if (dist < dMin) {
14                 dMin = dist;
15                 best = new PointPair(strip[i],
16                     strip[j]);
17             }
18         }
19     }
20     return best;
21 }
```

Listing 4. Mètode `processStrip` de la classe base.

3.5. Variant amb Buckets

La variant amb *buckets* (`DivideConquerClosestBucket`) intenta millorar el pas de processament de la franja. En lloc d'ordenar la franja per y amb cost $O(k \log k)$, distribueix els punts en cubetes (*buckets*) d'alçada $d_{\min}$ i comprova només la cubeta actual i la immediatament superior. Per l'argument de l'empaquetament, cada cubeta conté com a màxim 8 punts, de manera que el cost per punt és $O(1)$ i el cost total de la franja és $O(k)$.

Això reduiria la recurrència de $T(n) = 2T(n/2) + O(k \log k)$ a $T(n) = 2T(n/2) + O(k)$. Com que $k \leq n$, el primer cas dona $O(n \log^2 n)$ i el segon $O(n \log n)$.

3.5.1. Guarda: Fallback per a Buckets Dispersos. L'optimització per *buckets* requereix crear un array de cubetes indexat pel rang de coordenada y . El nombre de cubetes és:

$$\text{numBuckets} = \left\lceil \frac{y_{\max}}{d_{\min}} \right\rceil - \left\lceil \frac{y_{\min}}{d_{\min}} \right\rceil + 1 \quad (1)$$

Quan `numBuckets > k` (més cubetes que punts), l'array de cubetes seria excessivament dispers: el cost d'al·locació i la pèrdua de localitat de cache superen l'estalvi teòric. En aquest cas, l'algorisme fa *fallback* al processament per ordenació de la classe base (`super.processStrip()`).

3.5.2. Limitació Pràctica Fonamental. Durant l'anàlisi experimental s'ha descobert que, per al problema de la parella més propera, la guarda `numBuckets > k` s'activa sistemàticament per a totes les distribucions provades, fent que la variant amb *buckets* sempre caigui al *fallback* de la classe base.

La raó és fonamental: la distància mínima $d_{\min}$ decreix ràpidament a mesura que avança la recursió. Per a un núvol de punts amb rang R en l'eix y , el nombre de cubetes és `numBuckets` $\approx R/d_{\min}$, mentre que el nombre de punts a la franja és $k \leq n$. Com que $d_{\min} \ll R$ (la distància mínima és molt petita comparada amb el rang total), es té `numBuckets` $\gg k$ i la guarda s'activa.

Això significa que l'optimització per *buckets* **no aporta cap guany pràctic** per al problema de la parella més propera amb distribucions convencionals. És un resultat important perquè demostra que una optimització teòricament vàlida pot ser inaplicable en la pràctica degut a les propietats geomètriques del problema.

L'optimització per *buckets* seria útil en contextos on $d_{\min}$ és comparable al rang de la franja, com ara problemes on la distància mínima és gran (p. ex. conjunts de punts en graells regulars) o problemes de *range searching*.

3.6. Parella Més Llunyana

L'algorisme de la parella més llunyana (`FarthestPair`) combina dues tècniques:

3.6.1. Convex Hull: Algorisme d'Andrew. L'envolupant convexa es calcula amb l'algorisme de cadena monòtona d'Andrew [3], que construeix per separat les meitats inferior i superior de l'envolupant amb dos escaneigs lineals després d'ordenar per x . El cost total és $O(n \log n)$.

3.6.2. Rotating Calipers. Un cop obtinguda l'envolupant convexa amb h vèrtexs, la parella més llunyana és necessàriament un parell de vèrtexs antipodals [2]. L'algorisme dels *rotating calipers* recorre l'envolupant amb dos punters (un per cada línia de suport paral·lela) en temps $O(h)$, on $h \leq n$.

Com que el *convex hull* costa $O(n \log n)$ i els *rotating calipers* costen $O(h)$, el cost total és $O(n \log n)$.

3.7. Generació de Núvols de Punts

La classe `PointCloud` implementa les quatre distribucions amb la classe `java.util.Random`. Les distribucions gaussiana, exponencial i agrupada utilitzen mostreig per rebuig (*rejection sampling*) per garantir que tots els punts quedin dins del rang $[\min, \max]$.

Els paràmetres de cada distribució es defineixen com a arrays de `ParamSpec` (registre amb nom i valor per defecte), i els valors actuals s'encapsulen en `DistributionParams` (registre immutable amb còpia defensiva). El panell `DistributionParamPanel` genera dinàmicament els camps de text a partir de `ParamSpec[]`, de manera que afegir una nova distribució o un nou paràmetre no requereix canviar la vista.

3.8. Protocol de Benchmark

El benchmark segueix la metodologia de la diapositiva 125 dels apunts:

- 1) **Escalfament JIT:** abans del benchmark, s'executen tots els algorismes 10 vegades amb $n = 500$ per forçar la compilació JIT de totes les rutes de codi.
- 2) **Escalfament per execució:** les 3 primeres execucions de cada mesura es descarten per eliminar efectes de cache i compilació incremental.
- 3) **Mesura:** les 5 execucions següents es mesuren amb `System.nanoTime()` i es calcula la mitjana.

3.8.1. Paral·lisme de Gra Fi. La classe `Benchmark.runSeries` distribueix les tasques amb un `ExecutorService` amb tants fils com processadors disponibles. Les tasques es submeten amb granularitat fina: una tasca per cada parella (algorisme, n), en lloc d'una tasca per a cada n .

Això evita el coll d'ampolla de la versió serial: quan força bruta amb el n més gran triga centenars de mil·lisegons, els algorismes $O(n \log n)$ per al mateix n ja estan executant-se en nuclis separats en lloc d'esperar.

3.8.2. Ajust de Constants i Predicció. La classe `Predictor` ajusta constants multiplicatives als models teòrics amb mínims quadrats ponderats: per al model $t = a \cdot f(n)$ sense ordenada a l'origen, la constant és:

$$a = \frac{\sum_i t_i \cdot f(n_i)}{\sum_i f(n_i)^2} \quad (2)$$

Aquesta ponderació dóna més pes a les mesures amb n gran, on el comportament asimptòtic domina sobre l'overhead de la JVM.

3.8.3. CLI de Benchmark. A més de la interfície gràfica, s'ofereix una eina de línia de comandes (`BenchmarkCLI`) que permet executar benchmarks amb paràmetres configurables i sortida en format taula o CSV:

```
1 # Tots els algorismes, distribucio uniforme
2 java -cp target/classes \
3     com.serafinebot.p3.BenchmarkCLI
4
5 # D&C vs Bucket, gaussiana, CSV
6 java -cp target/classes \
7     com.serafinebot.p3.BenchmarkCLI \
8     -a dc,bucket -d GAUSSIAN --csv
9
10 # 3 execucions promitjades
11 java -cp target/classes \
12     com.serafinebot.p3.BenchmarkCLI -r 3
```

Listing 5. Exemples d'ús del CLI.

4. Anàlisi de Complexitat Teòrica

4.1. Força Bruta

L'algorisme examina totes les $\binom{n}{2}$ parelles:

$$T_{\text{brute}}(n) = \frac{n(n-1)}{2} \in O(n^2) \quad (3)$$

4.2. Divideix i Venceràs

La recurrència és:

$$T(n) = 2T(n/2) + C_{\text{strip}}(n) \quad (4)$$

Amb el processament clàssic de la franja (ordenació per y), el cost de la franja és $O(k \log k)$ on $k \leq n$, donant $C_{\text{strip}} = O(n \log n)$. Pel Teorema Mestre, amb $a = 2$, $b = 2$, $f(n) = n \log n$: com que $f(n) = \Theta(n \log n)$ i $n^{\log_b a} = n^1 = n$, es té $f(n) = \Omega(n^{1+\epsilon})$ només per $\epsilon = 0$, que no satisfà la condició del cas 3. Aplicant l'extensió del Teorema Mestre per al cas $f(n) = \Theta(n^c \log^k n)$ amb $c = 1$ i $k = 1$:

$$T(n) = O(n \log^2 n) \quad (5)$$

Amb *buckets* ($C_{\text{strip}} = O(k) = O(n)$), la recurrència $T(n) = 2T(n/2) + O(n)$ dóna $T(n) = O(n \log n)$ pel cas 2 del Teorema Mestre.

Com s'explica a la Secció 3.5.2, la variant amb *buckets* no s'activa en la pràctica, de manera que el cost real és $O(n \log^2 n)$. No obstant això, la diferència entre $O(n \log n)$ i $O(n \log^2 n)$ és mínima en la pràctica: per a $n = 50\,000$, $\log_2 n \approx 15.6$, de manera que el factor addicional $\log n$ és petit comparat amb la constant multiplicativa.

4.3. Parella Més Llunyana

El cost total és la suma del *convex hull* i els *rotating calipers*:

$$T_{\text{farthest}}(n) = O(n \log n) + O(h) = O(n \log n) \quad (6)$$

on h és el nombre de vèrtexs de l'envolupant convexa ($h \leq n$).

5. Anàlisi Experimental

Totes les mesures s'han realitzat amb el CLI de benchmark, amb 3 execucions independents promitjades, en una màquina amb processador Apple M1 (8 nuclis), 16 GB RAM i JDK 17.

5.1. Rendiment Global: Distribució Uniforme

La Taula 1 mostra els temps d'execució per a la distribució uniforme amb rang $[0, 1000]$.

Taula 1. TEMPS D'EXECUCIÓ (MS) PER A DISTRIBUCIÓ UNIFORME.

n	Bruta	D&C	D&C Bkt.	Llunyana
100	0.0049	0.0204	0.0149	0.0251
500	0.0962	0.2750	0.1648	0.1770
1000	0.3839	0.3764	0.4840	0.3931
2000	1.7004	1.5858	1.3526	0.5068
5000	10.3593	3.8483	3.7415	1.8985
10000	38.8218	5.1295	4.1223	2.5650
20000	142.5667	6.1267	6.1258	4.6799
50000	871.9498	15.0320	15.4075	8.4644

S'observa que:

- Força bruta domina per a $n \leq 500$ degut a la menor constant multiplicativa (no requereix ordenació).
- D&C supera força bruta a partir de $n \approx 1000$, on la constant multiplicativa de l'ordenació queda compensada per la creixença quadràtica de força bruta.
- D&C i D&C Bucket tenen rendiment pràcticament idèntic, confirmant que la variant amb *buckets* sempre fa *fallback* al processament clàssic.
- La parella llunyana és la més ràpida per a n grans, degut a la menor constant multiplicativa del *convex hull* + *rotating calipers*.

5.2. Comparació entre Distribucions

La Taula 2 compara els temps de D&C i la parella llunyana per a les quatre distribucions a $n = 50\,000$.

Taula 2. TEMPS (MS) PER ALGORISME I DISTRIBUCIÓ A $n = 50\,000$.

Distribució	Bruta	D&C	D&C Bkt.	Llunyana
Uniforme	871.95	15.03	15.41	8.46
Gaussiana	900.23	15.20	15.51	8.66
Exponencial	989.26	14.96	14.95	8.52
Agrupada (10)	905.89	14.03	14.33	8.77

Els temps de força bruta són pràcticament idèntics per a totes les distribucions, ja que l'algorisme sempre examina totes les parelles independentment de la geometria. D&C i la parella llunyana mostren variacions mínimes entre distribucions, confirmant que el cost asimptòtic domina sobre les propietats geomètriques.

5.3. Anàlisi de Creixement

Per verificar les complexitats teòriques, s'analitza el creixement del temps amb n .

5.3.1. Força Bruta: Creixement Quadràtic. Si $T(n) = a \cdot n^2$, el ràtio $T(2n)/T(n)$ hauria de ser ≈ 4 . La Taula 3 confirma aquest comportament.

Taula 3. RÀTIOS DE CREIXEMENT DE FORÇA BRUTA.

n	Temps (ms)	$T(2n)/T(n)$
1000	0.38	—
2000	1.70	4.43
5000	10.36	—
10000	38.82	3.75
20000	142.57	3.67
50000	871.95	—

Els ràtios s'acosten a 4, confirmant el comportament $O(n^2)$. Les desviacions (4.43 a $n = 2000$) es deuen a efectes de cache i JIT.

5.3.2. D&C: Creixement $O(n \log n)$. Si $T(n) = a \cdot n \log_2 n$, el ràtio $T(2n)/T(n)$ hauria de ser $\approx 2 \cdot \frac{\log_2(2n)}{\log_2(n)} = 2 \cdot (1 + 1/\log_2 n)$. Per exemple, a $n = 10\,000$: $2 \cdot (1 + 1/13.3) = 2.15$.

Taula 4. RÀTIOS DE CREIXEMENT DE D&C.

n	Temps (ms)	$T(2n)/T(n)$	Teòric
1000	0.38	—	—
2000	1.59	4.21	2.10
5000	3.85	—	—
10000	5.13	1.33	2.15
20000	6.13	1.19	2.08
50000	15.03	—	—

Els ràtios de D&C són irregulars per a n petits degut a l'overhead de la recursió i l'ordenació de la JVM, però s'estabilitzen per a n grans. El ràtio $T(50000)/T(20000) = 15.03/6.13 = 2.45$ és coherent amb el ràtio teòric de $2.5 \cdot (1 + 1/\log_2(20000)) \approx 2.67$ per a un factor de $50000/20000 = 2.5$.

5.4. D&C vs D&C Bucket: Rendiment Idèntic

La Taula 5 compara les dues variants de D&C amb valors de n fins a 500 000, confirmant que la variant amb *buckets* no aporta cap millora.

Taula 5. COMPARACIÓ D&C VS D&C BUCKET (MS), DISTRIBUCIÓ UNIFORME.

n	D&C	D&C Bkt.	Ràtio
1000	1.31	0.99	0.76
5000	2.39	2.37	1.00
10000	7.56	7.19	0.95
50000	16.31	17.25	1.06
100000	31.32	30.24	0.97
200000	59.76	59.99	1.00
500000	169.84	174.15	1.03

El ràtio Bucket/D&C oscil·la al voltant de 1.0 per a tots els valors de n , confirmant que les dues variants executen exactament el mateix codi (la variant amb *buckets* fa *fallback* a `super.processStrip()`) més l'overhead menor de la comprovació de la guarda.

5.5. Escalabilitat a N Grans

Per demostrar l'escalabilitat dels algorismes $O(n \log n)$, la Taula 5 mostra que D&C processa 500 000 punts en ≈ 170 ms, mentre que força bruta trigaria:

$$a_{\text{brute}} \cdot 500\,000^2 \approx 0.35 \cdot 10^{-6} \cdot 2.5 \cdot 10^{11} \approx 87\,500 \text{ ms} \approx 1.5 \text{ min} \quad (7)$$

Això representa un *speedup* de $\approx 515\times$.

5.6. Efectes de la Paral·lització del Benchmark

Durant el desenvolupament s'ha observat que l'execució paral·lela del benchmark pot introduir interferència entre algorismes. Quan un algorisme genera excessiva pressió sobre el *garbage collector* (p. ex. la variant amb *buckets* sense guarda, que al·loca arrays dispersos enormes), les pauses GC *stop-the-world* afecten tots els fils, incloent els que executen altres algorismes. Això es manifesta com un alentiment espuri de D&C quan s'executa en paral·lel amb una variant ineficient de *buckets*.

La solució adoptada (la guarda `numBuckets > k` amb *fallback*) elimina les al·locacions excessives i, per tant, la interferència entre algorismes.

6. Decisions de Disseny Notables

6.1. Template Method vs Duplicació

La decisió d'usar herència (Template Method) per a les dues variants de D&C té un compromís: el despatx virtual de `processStrip` afegeix un overhead mínim per crida. En la pràctica, el cost és insignificant (< 1 ns per crida) comparat amb el cost de la franja (> 1 μ s fins i tot per a franges petites), i s'estalvien ≈ 125 línies de codi duplicat.

6.2. distanceTo vs distanceSquaredTo

S'ha optat per utilitzar `distanceTo` (amb `Math.sqrt`) a tots els algorismes en lloc de

`distanceSquaredTo` (sense arrel quadrada). Tot i que `distanceSquaredTo` evita el cost de `sqrt`, això complicaria el codi: caldria mantenir `dMin` i `dMinSq` en paral·lel, ja que les comparacions de distància es poden fer amb quadrats però les comparacions geomètriques de la franja ($|x - x_{\text{mid}}| < d_{\text{min}}$ i $y_j - y_i < d_{\text{min}}$) requereixen la distància real.

En la pràctica, la diferència de rendiment és negligible perquè `Math.sqrt` és una instrucció nativa del processador (FSQRT en x86, FSQRT en ARM) que s'executa en pocs cicles.

6.3. Buffer Preallocat vs Al·locació per Nivell

El buffer preallocat `Point[n]` per a la franja elimina $O(\log n)$ al·locacions per crida a `find()`, reduint la pressió sobre el GC i millorant la localitat de cache. El compromís és que el buffer té mida n (la mida màxima possible de la franja és n al nivell arrel) tot i que els nivells inferiors utilitzen fraccions cada vegada més petites.

7. Conclusió i Opinió Personal

Referències

- [1] M. I. Shamos i D. Hoey, "Closest-point problems," *Proc. 16th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 151–162, 1975.
- [2] G. T. Toussaint, "Solving geometric problems with the rotating calipers," *Proc. IEEE MELECON*, pp. A10.02/1–4, 1983.
- [3] A. M. Andrew, "Another efficient algorithm for convex hulls in two dimensions," *Information Processing Letters*, vol. 9, no. 5, pp. 216–219, 1979.
- [4] E. Gamma, R. Helm, R. Johnson, i J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.